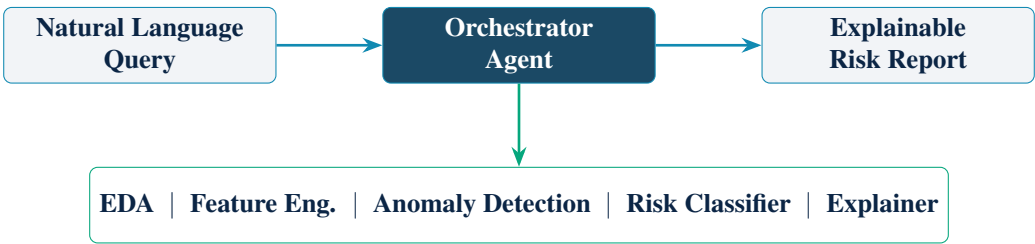


AI-Powered Suspicious Activity Detection

An Autonomous, Query-Aware Agentic System for AML Compliance

Solution Design & Technical Architecture Document

VIT Campus Hackathon — Problem Statement Response



“Not a fixed pipeline — a reasoning agent that decides what to run, on what data, and why.”

Contents

1	Executive Summary	2
2	Problem Restatement & Success Criteria	2
2.1	Core Challenge	2
2.2	What Makes This an <i>Agentic</i> Problem (not just an ML problem)	2
2.3	Success Criteria (mapped to problem statement)	3
3	High-Level System Architecture	3
3.1	Design Principles	3
4	Agent Orchestration Design	4
4.1	Step 1 – Intent & Entity Extraction	4
4.2	Step 2 – Dynamic Execution Plan	4
4.3	Step 3 – Tool Execution Loop (pseudocode)	4
4.4	Handling the Three Reference Queries	5
5	Tool Specifications	5
5.1	Data Loader & Preprocessing Tool	5
5.2	EDA Tool	6
5.3	Feature Engineering Tool	6
5.4	Anomaly / Suspicious-Pattern Detection Tool	6
5.5	Risk Classification Tool	6
5.6	Explanation Component	7
5.7	Escalation Policy Layer	7
6	Data & Schema Assumptions	7
6.1	Expected Input Schema	7
6.2	Sample Public Datasets for Demo/Testing	7
7	Technology Stack	8
8	Output Report Schema	8
9	Demo Walkthrough Plan	8
10	Evaluation Against Likely Judging Rubric	9
11	Risks & Mitigations	9
12	Conclusion	9

1 Executive Summary

Financial institutions are required to run Anti-Money Laundering (AML) programs, yet legacy rule-based transaction monitoring systems are notorious for excessive false positives and brittle logic that cannot adapt to evolving laundering techniques such as **structuring**, **smurfing**, and **layering**. This document presents the technical design for an **autonomous, LLM-driven agent** that replaces the fixed "load → EDA → detect → score" pipeline with a **dynamic, query-aware execution planner**.

Given a natural-language instruction (e.g. *"Find structuring patterns in the last 30 days"* or *"Is customer ID 4521 suspicious?"*), the agent:

1. Parses **intent**, **entities**, **filters**, and **target AML pattern** from the query;
2. Builds a **minimal, sufficient execution plan** – invoking only the tools the query actually needs;
3. Executes that plan over internal tools (EDA, Feature Engineering, Anomaly Detection, Risk Classification, Explanation);
4. Returns a **structured, judge-inspectable report** containing the plan itself, the flagged entities, risk levels, natural-language explanations, and escalation recommendations.

The system is built as a **tool-calling agent on top of a large language model** (via the Anthropic Claude API or an equivalent function-calling capable model), backed by a Python analytics layer (pandas, scikit-learn, PyOD) for the actual numerical work. The LLM is deliberately kept out of the numerical anomaly-scoring loop – it plans, orchestrates, and explains; it does not compute risk scores itself. This keeps the system auditable, deterministic where it needs to be, and still flexible enough to handle open-ended natural-language queries.

2 Problem Restatement & Success Criteria

2.1 Core Challenge

Traditional AML monitoring is **rule-based and sequential**: every transaction runs through the same fixed set of checks regardless of what a compliance officer actually asked for. This wastes compute, produces noisy alerts, and cannot answer targeted questions (*"is this one customer suspicious?"*) without re-running the entire pipeline.

2.2 What Makes This an *Agentic* Problem (not just an ML problem)

The system must reason about *which* tools to call, not just execute a fixed sequence

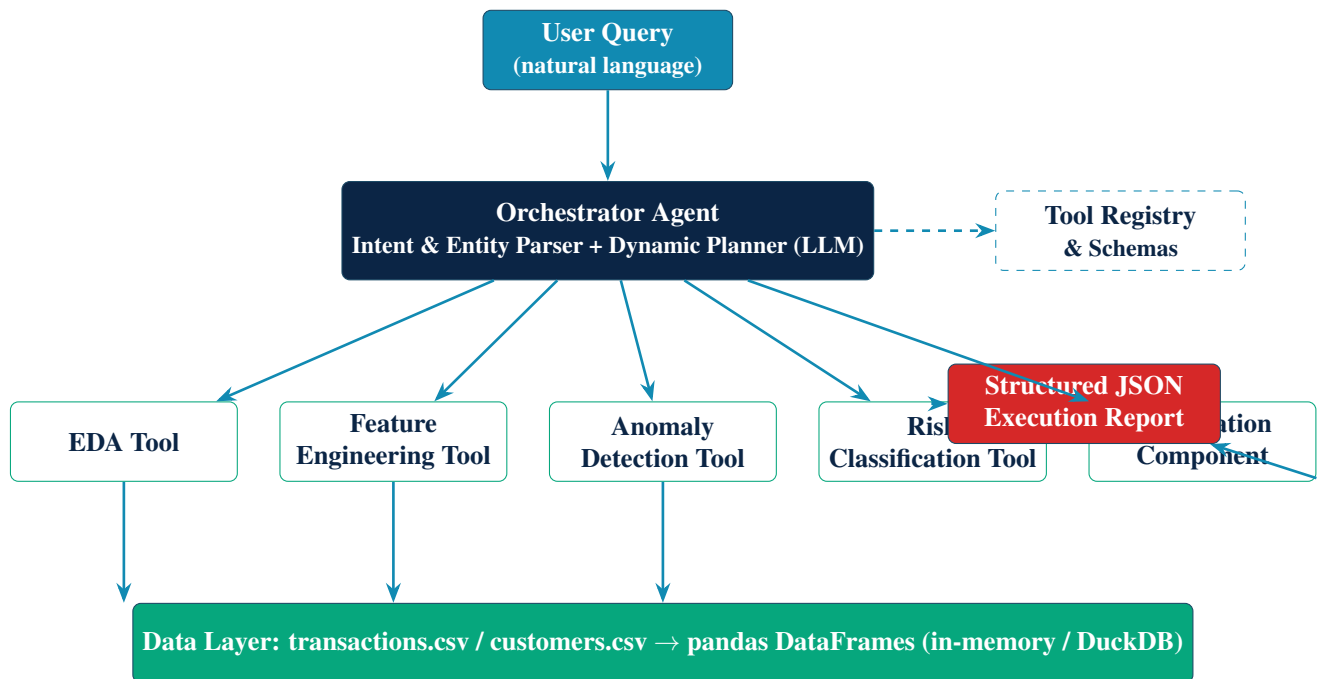
- A query about a **single customer** should never trigger full-dataset EDA.
- A query with an explicit **rule** ("10+ transactions under \$10,000") should skip ML anomaly detection entirely.
- A query about a **named AML pattern** ("structuring") should invoke only the feature engineering relevant to that pattern.

This is the differentiator the hackathon rubric is testing for: **planning + selective tool invocation**, not merely "run an isolation forest and show a dashboard."

2.3 Success Criteria (mapped to problem statement)

Requirement	How this design satisfies it
Automated EDA	Dedicated EDA Tool, invoked only when the plan calls for broad exploration
Anomaly / pattern detection	Hybrid ML + statistical + rule engine, pattern-specific feature sets
Risk score / flag per entity	Risk Classification Tool with configurable, context-aware thresholds
Explainability	Dedicated Explanation Tool using SHAP-style feature attributions + templated NL generation
Escalation recommendation	Deterministic policy layer mapping risk band → action
Non-sequential, query-aware agent	Central Orchestrator Agent with an LLM-based planner and a tool registry

3 High-Level System Architecture



3.1 Design Principles

1. **Plan before execution.** The LLM never touches raw transaction data directly; it produces a structured execution plan (a JSON object naming tools, arguments, and order) which a Python controller then executes deterministically.
2. **Tools are pure functions.** Each tool takes a filtered DataFrame (or query parameters) and returns structured output – no hidden state, no side effects. This makes the system testable and the judge's job of inspecting "what happened" trivial.
3. **Selective invocation, not selective silence.** If a tool is skipped, the report explicitly states *why* it was skipped (e.g. *"EDA skipped: query scoped to a single customer ID"*), which is exactly what the rubric asks the agent to expose.
4. **Numbers come from code, words come from the LLM.** Risk scores, thresholds, and aggregations are computed in deterministic Python/ML code; the LLM's role is limited to (a) planning and (b)

turning structured results into fluent, query-grounded explanations. This avoids LLM hallucination of numeric risk values.

4 Agent Orchestration Design

4.1 Step 1 – Intent & Entity Extraction

The first LLM call converts free text into a structured `QuerySpec`. This is done via constrained function-calling / structured output (JSON schema), not free-form text.

```

1 {
2   "intent": "pattern_detection | aggregation_rule | entity_lookup | broad_exploration",
3   "aml_pattern": "structuring | smurfing | layering | rapid_cashout | null",
4   "filters": {
5     "date_range": {"start": "YYYY-MM-DD", "end": "YYYY-MM-DD"},
6     "customer_id": null,
7     "segment": null,
8     "country": null,
9     "transaction_type": null
10  },
11  "explicit_rule": {
12    "condition": "count(transactions) >= 10 AND amount < 10000",
13    "present": true
14  },
15  "requires_ml_anomaly_detection": false,
16  "requires_full_eda": false
17 }
```

Listing 1: QuerySpec schema produced by the LLM from user input

4.2 Step 2 – Dynamic Execution Plan

The `QuerySpec` is passed to a planning function (still LLM-driven, but constrained to choose only from the registered tool list) which emits an ordered `ExecutionPlan`:

```

1 {
2   "plan_id": "plan_0091",
3   "reasoning": "Query targets a specific AML pattern (structuring) with a time filter; broad
4     EDA is unnecessary. Structuring detection needs frequency + amount-deviation features
5     and ML/statistical scoring, not a fixed dollar-threshold rule.",
6   "steps": [
7     {"tool": "data_loader", "args": {"date_range": ["2026-06-24", "2026-07-24"]}},
8     {"tool": "feature_engineering", "args": {"feature_set": "structuring"}},
9     {"tool": "anomaly_detection", "args": {"method": "hybrid", "target_pattern":
10       "structuring"}},
11     {"tool": "risk_classification", "args": {"scheme": "pattern_aware"}},
12     {"tool": "explanation", "args": {"tie_to_query": true}}
13   ],
14   "skipped_tools": [
15     {"tool": "eda_tool", "reason": "Query is pattern-targeted with explicit time filter;
16       full-dataset profiling adds no value here."}
17   ]
18 }
```

Listing 2: Example ExecutionPlan for "Find structuring patterns in the last 30 days"

A Python `AgentController` then walks steps in order, calling the corresponding Python tool function with validated arguments, and short-circuits/retries if a tool raises a schema-validation error. This separation – *LLM plans, Python executes* – is what keeps the agent both flexible and safe to run on real financial data.

4.3 Step 3 – Tool Execution Loop (pseudocode)

```

1 class AgentController:
2     def __init__(self, tool_registry, llm_client):
3         self.tools = tool_registry          # name -> callable
4         self.llm = llm_client
```

```

5
6  def run(self, user_query: str, df_transactions, df_customers):
7      spec = self.llm.extract_query_spec(user_query) # Step 1
8      plan = self.llm.build_execution_plan(spec, list(self.tools)) # Step 2
9
10     context = {"transactions": df_transactions,
11               "customers": df_customers,
12               "query_spec": spec}
13
14     results = {}
15     for step in plan["steps"]:
16         tool_fn = self.tools[step["tool"]]
17         results[step["tool"]] = tool_fn(context, **step["args"])
18         context.update(results) # pass forward for later tools
19
20     report = self.llm.generate_report(user_query, spec, plan, results)
21     return report

```

Listing 3: Core orchestration loop

4.4 Handling the Three Reference Queries

Query	Detected Intent	Tools Invoked (in order)
"Find structuring patterns in the last 30 days"	pattern_detection, aml_pattern=structuring, time filter present	data_loader (time-filtered) → feature_engineering (structuring set) → anomaly_detection → risk_classification → explanation
"Which customers made 10+ transactions under \$10,000?"	aggregation_rule, plicit_rule present, quires_ml=false	ex- data_loader → rule_engine re- (count + threshold aggregation) → risk_classification → explana- tion (feature_engineering & ML anomaly_detection skipped)
"Is customer ID 4521 suspicious?"	entity_lookup, tomer_id=4521	cus- data_loader (single-customer slice) → feature_engineering (lightweight, entity-scoped) → risk_classification (on-demand score / retrieve cached flag) → explanation (full EDA skipped)

5 Tool Specifications

5.1 Data Loader & Preprocessing Tool

Purpose: Load raw transaction/customer data and apply only the preprocessing the plan requires (type coercion, timezone normalization, currency conversion, dedup) – never a one-size-fits-all cleaning pass.

Input: file path(s) / DB connection, filters from QuerySpec.

Output: filtered `pandas.DataFrame` + a `preprocessing_log` (rows dropped, nulls imputed) for auditability.

5.2 EDA Tool

Purpose: Broad exploratory profiling – transaction volume over time, amount distributions, customer segment breakdowns, correlation heatmaps.

Invoked when: `intent = broad_exploration` or the query has no specific entity/pattern focus (e.g. "Analyse this dataset for suspicious activity").

Skipped when: query is entity-scoped, pattern-scoped with explicit filters, or rule-based.

Techniques: pandas-profiling-style summary stats, matplotlib/plotly charts (amount histograms, time-series volume, degree-distribution of counterparties), z-score outlier flags as a first-pass signal.

5.3 Feature Engineering Tool

Purpose: Construct AML-relevant, pattern-specific features on demand rather than one giant fixed feature table.

Feature families (built only as needed):

- *Structuring / smurfing*: count of transactions just below reporting threshold (e.g. \$10,000) per rolling 24h/7d/30d window; ratio of near-threshold transactions to total.
- *Velocity*: transaction frequency per customer per time bucket; inter-transaction time deltas.
- *Layering*: counterparty network depth / hop count; fan-in / fan-out ratios; rapid pass-through balance (funds in $\approx$ funds out within short window).
- *Rapid cash-out*: time between deposit and withdrawal/ATM cash-out; cash-out ratio.
- *Amount deviation*: z-score of transaction amount vs. that customer's historical mean/std; deviation from peer-group (segment) norms.

5.4 Anomaly / Suspicious-Pattern Detection Tool

Purpose: Score transactions/customers as anomalous using the method appropriate to the query – not always ML.

Methods available (agent chooses):

- **Rule engine** – direct threshold/count logic (e.g. "10+ txns \$<\$10,000"); used when the query itself states a concrete rule.
- **Statistical** – z-score / IQR outlier detection on engineered features; fast, explainable, good baseline.
- **ML-based unsupervised** – IsolationForest and Local Outlier Factor (via scikit-learn / PyOD) on the pattern-specific feature set; used for structuring/layering where no single rule captures the pattern.
- **Hybrid** – combine rule pre-filtering (cheap) with ML scoring on the surviving subset (expensive), reducing compute and false positives simultaneously.

Output: an `anomaly_score` (0–1) per entity plus the top contributing features (via IsolationForest path-length attribution or simple feature z-scores).

5.5 Risk Classification Tool

Purpose: Convert raw anomaly scores / rule hits into a business-facing **Low / Medium / High** risk band using context-appropriate thresholds (a single fixed cutoff is exactly the brittleness the problem statement is complaining about).

Approach: percentile-based thresholds computed *within the filtered cohort* (e.g. top 5% = High, next 15% = Medium) blended with hard business rules (e.g. any exact rule match $\Rightarrow$ minimum Medium,

any prior SAR filing on the entity $\Rightarrow$ minimum High).

5.6 Explanation Component

Purpose: Turn structured scores + top contributing features into a concise, query-grounded natural-language reason – this is an LLM call, but constrained to only the numeric facts passed in (no hallucinated figures).

Example: *"Customer 4521 was flagged Medium risk: 6 deposits of \$9,200–\$9,800 within 48 hours (just under the \$10,000 reporting threshold), a pattern consistent with structuring. This matches your query's focus on structuring activity."*

5.7 Escalation Policy Layer

Purpose: Deterministic mapping from risk band $\rightarrow$ recommended action, so this step is never left to LLM judgment.

Risk Band	Recommended Action	Rationale shown to reviewer
Low	Monitor	No further action; keep in rolling watch list
Medium	Flag for review	Analyst review within SLA (e.g. 3 business days)
High	Report (SAR draft)	Auto-draft Suspicious Activity Report for compliance sign-off

6 Data & Schema Assumptions

6.1 Expected Input Schema

```

1 transactions.csv
2   transaction_id, customer_id, timestamp, amount, currency,
3   transaction_type (deposit/withdrawal/transfer/atm),
4   counterparty_id, counterparty_country, channel
5
6 customers.csv
7   customer_id, name_hash, account_open_date, segment,
8   country, occupation, risk_rating_existing, kyc_flags

```

Listing 4: Assumed minimal transaction schema (csv/parquet)

If the hackathon-provided dataset differs, the Data Loader Tool's column-mapping layer normalizes it to this internal schema before any other tool runs – this keeps the rest of the agent dataset-agnostic.

6.2 Sample Public Datasets for Demo/Testing

- IBM AML synthetic transaction dataset (Kaggle) – has labelled laundering patterns.
- PaySim mobile-money synthetic fraud dataset (Kaggle) – good for structuring/rapid-cashout simulation.
- Synthetic data generator (custom script) to inject known structuring/smurfing cases for a guaranteed live demo of "true positives."

7 Technology Stack

Layer	Choice
Agent reasoning / planning	Anthropic Claude API (tool-use / function-calling) or OpenAI function-calling; structured-output mode for QuerySpec & ExecutionPlan
Orchestration glue	Python AgentController (custom, thin – avoids heavy framework lock-in); optionally LangGraph if a graph-based state machine is preferred
Data handling	pandas, DuckDB for larger-than-memory querying
Anomaly detection	scikit-learn (IsolationForest, LocalOutlierFactor), PyOD (ensemble anomaly detectors)
Explainability	SHAP for tree-based model attributions; simple z-score attribution for statistical path
Visualization	plotly (interactive, embeddable in report), matplotlib for static exports
API / demo layer	FastAPI backend exposing /query endpoint; simple React or Streamlit front-end for judges to type a query and see the live plan + report
Storage	SQLite/DuckDB for demo persistence of flags and prior SAR-equivalent decisions (supports the "prior SAR $\Rightarrow$ minimum High" rule)

8 Output Report Schema

This is the single structured object the front-end renders and the judges inspect directly.

```

1 {
2   "user_query": "Find structuring patterns in the last 30 days",
3   "query_spec": { "...": "as in Section 4.1" },
4   "execution_plan": { "...": "as in Section 4.2, including skipped_tools + reasoning" },
5   "flagged_entities": [
6     {
7       "customer_id": "4521",
8       "risk_score": 0.87,
9       "risk_band": "High",
10      "aml_pattern_matched": "structuring",
11      "top_contributing_features": [
12        {"feature": "near_threshold_txn_count_7d", "value": 6, "z_score": 3.1},
13        {"feature": "avg_txn_amount_deviation", "value": 2.4, "z_score": 2.4}
14      ],
15      "explanation": "Customer 4521 made 6 deposits of $9,200-$9,800 within 7 days...",
16      "recommended_action": "Report (SAR draft)"
17    }
18  ],
19  "summary_metrics": {
20    "total_transactions_scanned": 48213,
21    "entities_flagged": 17,
22    "high_risk": 3, "medium_risk": 9, "low_risk": 5
23  },
24  "charts": ["amount_distribution.png", "flagged_timeline.png"]
25 }
```

Listing 5: Final ExecutionReport returned to the user

9 Demo Walkthrough Plan

1. **Setup:** Load synthetic dataset with 3–4 injected structuring/smurfing cases and one clean high-volume customer (to prove low false-positive behaviour).

2. **Query 1 (broad):** *"Analyse this dataset for suspicious activity"* → agent runs full EDA + all detection tools; show the plan explicitly choosing every tool.
3. **Query 2 (pattern-targeted):** *"Find structuring patterns in the last 30 days"* → show EDA being *skipped* in the plan, and the reasoning string explaining why.
4. **Query 3 (rule-based):** *"Which customers made 10+ transactions under \$10,000?"* → show ML anomaly detection being skipped, rule engine used directly.
5. **Query 4 (entity lookup):** *"Is customer ID 4521 suspicious?"* → show single-entity slice, fast on-demand scoring, no EDA/no full dataset scan.
6. **Judge takeaway:** the same underlying tools, four completely different execution paths, all visible in the plan JSON – this directly demonstrates the "not a fixed pipeline" requirement.

10 Evaluation Against Likely Judging Rubric

Likely Criterion	Coverage
Agentic behaviour / dynamic planning	Central to design (Sections 4, 4.4) – explicit plan JSON with reasoning and skipped-tool justification
Detection accuracy	Hybrid rule + statistical + ML approach, tuned per AML pattern
Explainability	Dedicated, constrained-LLM explanation tool tied to query intent
Escalation logic	Deterministic policy table, fully auditable
Engineering quality / modularity	Pure-function tools, typed schemas, tool registry pattern
Judge inspectability	Full plan + reasoning + skipped tools + metrics returned in one JSON payload

11 Risks & Mitigations

Risk	Mitigation
LLM misclassifies intent, invokes wrong tools	Few-shot examples in the planning prompt (the three reference queries + more); a validation layer rejects plans referencing unregistered tools or missing required args
LLM hallucinates numeric risk values in explanations	Explanation prompt is given <i>only</i> the already-computed numbers and explicitly instructed to restate, not invent, figures; a regex/number-match post-check flags mismatches
Latency from multiple LLM calls	Cache QuerySpec → Plan mappings for repeated query shapes; run steps 1 and 2 in a single structured-output call where possible
Class imbalance in ML anomaly detection	Use unsupervised methods (Isolation Forest/LOF) that don't require labeled fraud examples; validate with injected synthetic positives
Data privacy of customer PII during demo	Hash/pseudonymize customer identifiers in all logs and displayed output

12 Conclusion

This design directly answers the hackathon's central ask: an agent that **reasons about what to do**, rather than a static analytics pipeline with an AI label pasted on top. By keeping numeric computation in deterministic, testable Python/ML code and reserving the LLM for planning and natural-language explanation,

the system stays both **flexible** (handles arbitrary natural-language AML queries) and **trustworthy** (every number in the final report is traceable to a specific tool call the judge can inspect). The modular tool registry also means new AML pattern detectors (e.g. trade-based laundering, crypto mixing) can be added later without touching the orchestration layer at all.